MLOPS WITH CLOUD PLATFORMS

© 2011-25 National University of Singapore. All Rights Reserved.

Topics

- Introduction to MLOps
- Pipeline design
- Monitoring and Logging
- **MLOps with Cloud platforms**

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Popular cloud platforms

- Amazon Web Service (AWS)
- Azure
- Google Cloud Platform (GCP)

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Setting up an S3 bucket**
  - Store ML runs by syncing local mlruns folder
  - Centralized Model Registry
- **Building and Pushing the MLFlow Docker Container**
  - Docker container is built and pushed to **AWS Elastic Container Registry (ECR)** using the mlflow sagemaker build-and-push-container command

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Deploying model to AWS SageMaker**
  - Get the image from ECR by specifying the image url
  - Get the model information from S3 bucket by specifying the model uri
  - deployment is initiated using the **mlflow.sagemaker.deploy function** with the mode set to mfs.DEPLOYMENT_MODE_CREATE
  - status of the SageMaker endpoint can be monitored in the AWS console, changing from "creating" to **"InService"** upon successful deployment

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Making Prediction**
  - predictions are made using **boto3**, the AWS SDK for Python
  - **batch querying** is necessary for large datasets as there is a limitation on number of samples per request
- **Switch model**
  - mlflow.sagemaker.deploy again, but setting the mode to **mfs.DEPLOYMENT_MODE_REPLACE** and providing the new_model_uri
  - Don’t need to delete and recreate

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

AWS Platform

- **Remove deployed model**
  - **mlflow.sagemaker.delete function** is used to remove the endpoint and the deployed model, requiring the app_name and region
  - Remove deployed model when you are not in use. It costs $$$!

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Install the azureml-sdk in your Python environment**
- **Create an Azure workspace**
- **Setting up an BLOB**
  - Store ML runs by syncing local mlruns folder
  - Centralized Model Registry
- **Building and Pushing the MLFlow Docker Container**
  - Deploy to Azure Container Instances (ACI)

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Deploying model to Azure**
  - ACI webservice specifically for debugging or testing models in a development stage
  - Deploy an ACI webservice instance based on the created model image using Webservice.deploy_from_image
  - MLFlow provides built-in support to deploy models directly to production using a container instance

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Azure Platform

- **Making Prediction**
  - predictions are made by querying the production endpoint's scoring_uri with appropriately scaled data
  - **batch querying** is necessary for large datasets as there is a limitation on number of samples per request
- **Switch model**
  - MLFlow does not provide explicit functionality to switch models on an existing Azure service
  - Requires to delete and recreate

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- MLFlow does not provide explicit support for deploying in Google Cloud
- Install Google Cloud SDK
- **Setting up an Google Storage bucket**
  - To store the contents of your mlruns folders
- **VM (Setup)**
  - To host the model
  - Installing **Conda** (essential for MLFlow to reconstruct environments) and **MLFlow** within the VM

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- **Deploying model to GCP**
  - model is served by executing the mlflow models serve command directly on the VM
- **Making Prediction**
  - HTTP POST requests with JSON data (using application/json) to the VM's external IP address on port 5000(for example)
  - Batch querying is implemented for larger datasets, breaking them into smaller samples due to request limits

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

GCP Platform

- **Switch model**
  - Doesn’t support swtich directly.
  - Cancel the current ***mlflow models serve*** command
  - Re-run the ***mlflow models serve command*** is re-run with the new model's URI (run ID)
- **Remove model**
  - Cancel the current ***mlflow models serve*** command

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Comparison Table

Category	AWS	Azure	Google Cloud
ML Platform	SageMaker	Azure Machine Learning	Vertex AI
Data Storage	S3	Blob Storage	Cloud Storage
Data Pipeline	Glue, Data Pipeline, Step Functions	Data Factory	Dataflow, Composer (Airflow)
Training Compute	SageMaker Training, EC2	Azure ML Compute	Vertex Training / GKE
Experiment Tracking	SageMaker Experiments	Azure ML Run History	Vertex ML Metadata
Model Registry	SageMaker Model Registry	Azure Model Registry	Vertex Model Registry
Deployment	SageMaker Endpoint / Lambda	AKS / Azure Functions	Vertex Prediction / Cloud Run
CI/CD	CodePipeline + CodeBuild	Azure DevOps / GitHub Actions	Cloud Build + Cloud Deploy
Monitoring	SageMaker Model Monitor, CloudWatch	Azure Monitor + ML Monitor	Vertex Monitoring, Cloud Monitoring
Drift Detection	SageMaker Model Monitor	Custom with Azure Monitor	Vertex Model Monitoring
Security	IAM, KMS, PrivateLink	RBAC, Key Vault	IAM, VPC, Data Loss Prevention

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Best Practices

- Use versioning for datasets, models, and code.
- Implement access control roles with fine-grained access.
- Schedule model retraining via Serverless such as AWS Step Function to be more cost effective.
- Store metrics and logs in a log repository and storage such as S2 or Blob storage for audits.
- Use multi-account strategy for dev/test/prod segregation.

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Scale the ML System – Pipeline Scaling

What's wrong monolith pipeline ?

- Coupling
  - Change in data breaks deployment
- Fragility
  - One failure blocks entire pipeline
- Slow delivery
  - Every update requires full redeploy
- No isolation
  - One team's experiment impacts others
- Poor scalability
  - Training scale ≠ serving scale
- Governance chaos
  - No clear ownership or audit trail

ML systems scale *non-linearly* — pipelines must scale **independently**, not together.

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Scale the ML System – Cloud Native Architecture

Independent scaling of pipeline stages (the core benefit)

ML Stage	Cloud-native scaling
Data ingestion	Scales with data volume
Feature engineering	Parallel batch / streaming
Training	Elastic GPU/CPU pools
Inference	Horizontal autoscaling
Monitoring	Event-driven, async

Each stage is its **own service/workload.**
Training can scale to 1,000 GPUs while inference stays lightweight—and neither blocks the other.

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Containers = reproducibility + horizontal scale

Why containers matter for ML ?

- Same image → dev, test, prod
- Immutable environments
- No “works on my laptop” issues

At scale

- Training jobs run as **short-lived containers**
- Inference runs as **long-lived services**
- Easy to replicate, restart, rollback

Containers turn ML from “scripts” into **deployable units of scale**

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Orchestration enables elastic ML (Kubernetes / managed services)

Automatic scheduling

- Train 1 model → 1 pod
- Train 100 models → 100 pods

Autoscaling

- Scale up on demand
- Scale down when idle
- Pay only for what you use

Isolation

- Per-team namespaces
- Resource quotas
- Fault containment

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Control plane vs data plane separation (scaling without chaos)

Cloud-native platforms enforce this separation **by design**.

Control Plane (governance & coordination)

- Pipelines
- Metadata
- Policies
- Model registry
- CI/CD
- Scales **logically**, not computationally.

Data Plane (execution & compute)

- Training jobs
- Batch inference
- Online serving
- 👉 Scales **elastically** with demand.

© 2011-24 National University of Singapore. All Rights Reserved.

MLOps with platforms

Event-driven & asynchronous pipelines

Cloud-native ML pipelines are **reactive**, not linear.

Examples

- New data → trigger retraining
- Model drift → trigger evaluation
- Approval → trigger deployment

Benefits

- No blocking stages
- No idle waiting
- No manual coordination

This is how ML pipelines survive at 1000+ models.

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- MLFlow and deployment with 3 most popular cloud platforms and best practices
- Scale the ML Pipeline using Cloud Native architecture

© 2011-24 National University of Singapore. All Rights Reserved.